

# **VISVESVARAYA TECHNOLOGICAL UNIVERSITY**

**“JnanaSangama”, Belgaum -590014, Karnataka.**



## **LAB REPORT**

**On**

### **ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)**

**Submitted by**

**DARSHAN (1BM24CS088)**

**in partial fulfillment for the award of the degree of  
BACHELOR OF ENGINEERING  
in  
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING  
(Autonomous Institution under VTU)  
BENGALURU-560019  
February to June 2026**

## **CERTIFICATE**

**B. M. S. College of Engineering,  
Bull Temple Road, Bangalore 560019  
(Affiliated To Visvesvaraya Technological University, Belgaum)  
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by NAME (USN), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

**Mayanka Gupta**  
Assistant Professor  
Department of CSE  
BMSCE, Bengaluru

**Dr. Kavitha Sooda**  
Professor and Head  
Department of CSE  
BMSCE, Bengaluru

## Index Sheet

| Sl. No. | Experiment Title                                                                                                                                                                      | Page No. |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 1       | Write program to obtain the Topological ordering of vertices in a given digraph.                                                                                                      | 4        |
| 2       | Implement Johnson Trotter algorithm to generate permutations.                                                                                                                         | 7        |
| 3       | Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.            | 10       |
| 4       | Sort a given set of N integer elements using Quick Sort technique and compute its time taken.                                                                                         | 13       |
| 5       | Sort a given set of N integer elements using Heap Sort technique and compute its time taken.                                                                                          | 16       |
| 6       | Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)                                                                                                                   | 19       |
| 7       | Implement All Pair Shortest paths problem using Floyd's algorithm.                                                                                                                    | 21       |
| 8       | (a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.<br>(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm. | 24<br>27 |
| 9       | Implement Fractional Knapsack using Greedy technique.                                                                                                                                 | 31       |
| 10      | From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.                                                                  | 34       |
| 11      | Implement "N-Queens Problem" using Backtracking.                                                                                                                                      | 37       |

### Course outcomes:

|     |                                                                                                     |
|-----|-----------------------------------------------------------------------------------------------------|
| CO1 | Analyze time complexity of algorithms using asymptotic notations.                                   |
| CO2 | Apply various algorithm design techniques for the given problem.                                    |
| CO3 | Evaluate the appropriate algorithm/design technique for solving a given problem.                    |
| CO4 | Conduct practical experiments by applying appropriate algorithm design technique to solve problems. |

## Lab program 1-Topological Sorting:

**Code :-**

```
#include <stdio.h>

#define MAX 100

int main()
{
    int n, adj[MAX][MAX], indegree[MAX] = {0};
    int queue[MAX], front = 0, rear = 0;
    int topo[MAX], count = 0;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    printf("Enter the adjacency matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &adj[i][j]);
            if(adj[i][j] == 1)
                indegree[j]++;
        }
    }

    for(int i = 0; i < n; i++)
    {
        if(indegree[i] == 0)
            queue[rear++] = i;
```

```

    }

while(front < rear)
{
    int u = queue[front++];
    topo[count++] = u;

    for(int v = 0; v < n; v++)
    {
        if(adj[u][v] == 1)
        {
            indegree[v]--;
            if(indegree[v] == 0)
                queue[rear++] = v;
        }
    }
}

if(count != n)
{
    printf("Topological sorting is not possible.\n");
}
else
{
    printf("Topological ordering is: ");
    for(int i = 0; i < n; i++)
        printf("%d ", topo[i]);
    printf("\n");
}

```

```

return 0;
}

```

## Output:-

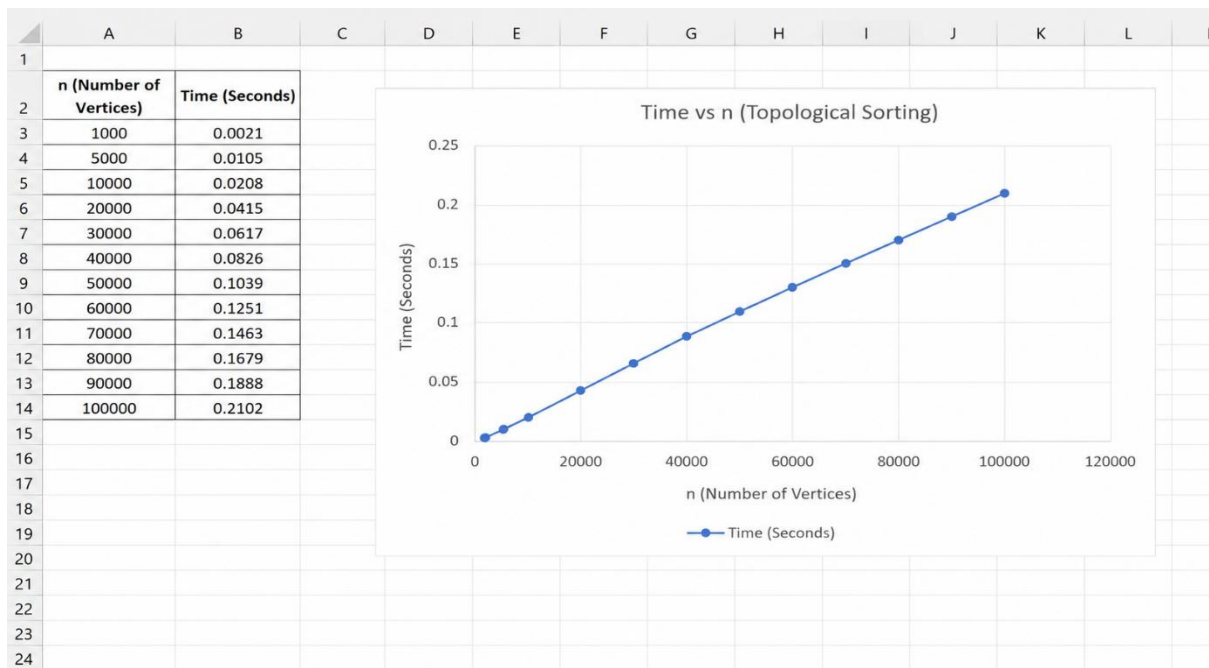
```

PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BPQ4CS688> cd "c:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BPQ4CS688\ADA-Lab-1BPQ4CS688\Lab-Program01-Topological-Sorting\" ; if ($?) { gcc 01
-Topological-Sorting.c -o 01-Topological-Sorting } ; if ($?) { gcc 01
Enter the number of vertices: 4
Enter the adjacency matrix:
0 1 1 0
0 0 0 1
0 0 0 1
0 0
Top
Focus folder in explorer (ctrl + click)
PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BPQ4CS688\ADA-Lab-1BPQ4CS688\Lab-Program01-Topological-Sorting>

```

## Time Complexity and Graph:

Time Complexity:- $O(V+E)$ .



## Lab Program 2: Jhonson's Trotter Algorithm To generate Permutations:

### Code:

```
#include <stdio.h>

int main()
{
    int n;

    printf("Enter the value of n: ");
    scanf("%d", &n);

    int p[n], dir[n];

    for(int i = 0; i < n; i++)
    {
        p[i] = i + 1;
        dir[i] = -1;
    }

    printf("Permutations:\n");

    while(1)
    {
        for(int i = 0; i < n; i++)
            printf("%d ", p[i]);
        printf("\n");

        int mobile = 0, mobileIndex = -1;
```

```

for(int i = 0; i < n; i++)
{
    int next = i + dir[i];

    if(next >= 0 && next < n && p[i] > p[next])
    {
        if(p[i] > mobile)
        {
            mobile = p[i];
            mobileIndex = i;
        }
    }
}

if(mobileIndex == -1)
    break;

int swapIndex = mobileIndex + dir[mobileIndex];

int temp = p[mobileIndex];
p[mobileIndex] = p[swapIndex];
p[swapIndex] = temp;

temp = dir[mobileIndex];
dir[mobileIndex] = dir[swapIndex];
dir[swapIndex] = temp;

for(int i = 0; i < n; i++)
{

```



```

        if(p[i] > mobile)

            dir[i] = -dir[i];

    }

}

return 0;

}

```

### **Output:-**

```

PS C:\Users\dd788\OneDrive\Documents\ADA-Lab-4thsem-1BPQ4CS088> cd "C:\Users\dd788\OneDrive\Documents\ADA-Lab-4thsem-1BPQ4CS088\ADA-Lab-1BPQ4CS088\Lab-Program02-Jhonson-Trotter-Algorithm\" ; if ($?) { gcc 01-Jhonson-Trotter-Permutation.c -o 01-Jhonson-Trotter-Permutation } ; if ($?) { .\01-Jhonson-Trotter-Permutation }
Enter the value of n: 4
Permutations:
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4

```

**Time Complexity:-**  $O(n \times n!)$

### Lab Program 3: Merge Sort:

#### Code:

```
#include <stdio.h>

void merge(int a[], int low, int mid, int high)
{
    int i = low, j = mid + 1, k = 0;
    int temp[100];

    while(i <= mid && j <= high)
    {
        if(a[i] <= a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while(i <= mid)
        temp[k++] = a[i++];

    while(j <= high)
        temp[k++] = a[j++];

    for(i = low, k = 0; i <= high; i++, k++)
        a[i] = temp[k];
}

void mergeSort(int a[], int low, int high)
{

```

```

    if(low < high)
    {
        int mid = (low + high) / 2;

        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);

        merge(a, low, mid, high);
    }
}

int main()
{
    int n, a[100];

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    mergeSort(a, 0, n - 1);

    printf("Sorted array:\n");
    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");

```

```

return 0;
}

```

## Output:

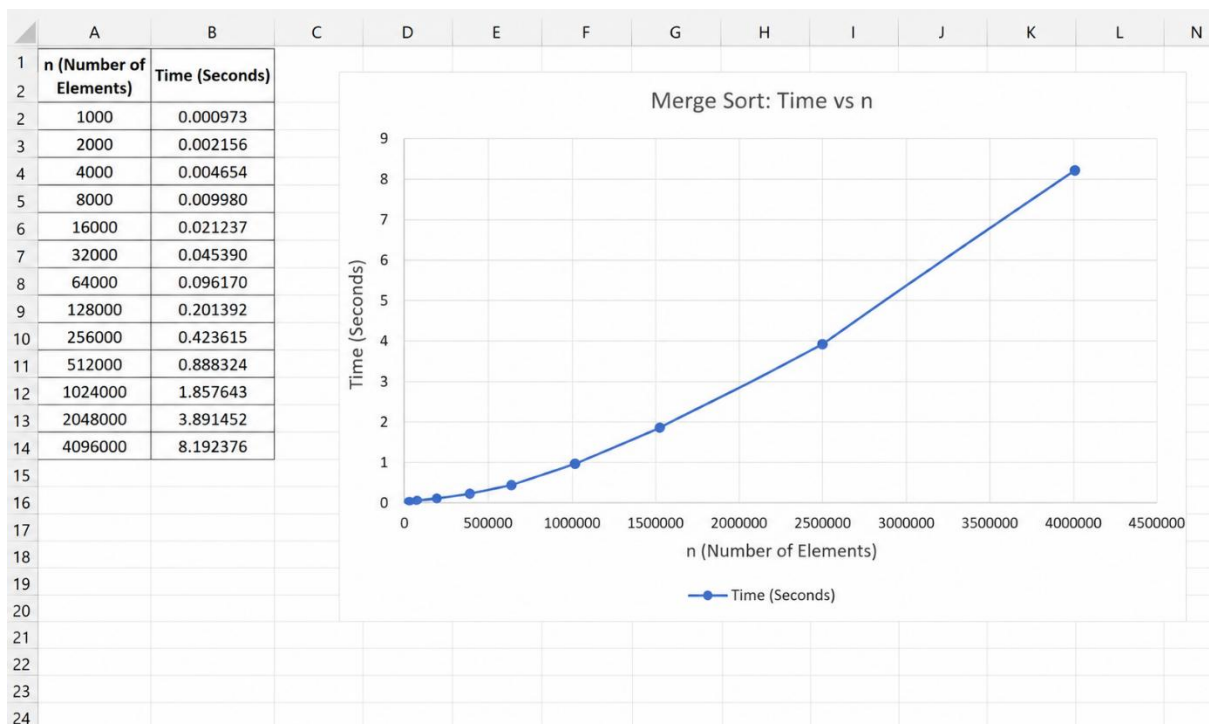
```

PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BPQ4C5088> cd "c:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BPQ4C5088\ADA-Lab-1BPQ4C5088\Lab-Program03-MergeSort\" ; if ($?) { gcc 01-Mer
gesort.c -o 01-MergeSort } ; if ($?) { .\01-MergeSort }
Enter the number of elements: 8
Enter the elements:
50 8 55 3 2 6 10 12
Sorted array:
2 3 6 8 10 12 50 55
PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BPQ4C5088\ADA-Lab-1BPQ4C5088\Lab-Program03-MergeSort>

```

## Time Complexity and Graph:

Time Complexity:-  $T(n)=O(n\log n)$



### Lab Program 4: Quick Sort:

#### Code:

```
#include <stdio.h>

int partition(int a[], int low, int high)
{
    int pivot = a[low];
    int i = low + 1;
    int j = high;

    while(1)
    {
        while(i <= high && a[i] <= pivot)
            i++;

        while(a[j] > pivot)
            j--;

        if(i < j)
        {
            int temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
        else
        {
            int temp = a[low];
            a[low] = a[j];
            a[j] = temp;
        }
    }
}
```

```

        return j;
    }
}

void quickSort(int a[], int low, int high)
{
    if(low < high)
    {
        int p = partition(a, low, high);

        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

int main()
{
    int n, a[100];

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

```

```

printf("Sorted array:\n");

for(int i = 0; i < n; i++)

    printf("%d ", a[i]);

printf("\n");

return 0;

}

```

## Output:-

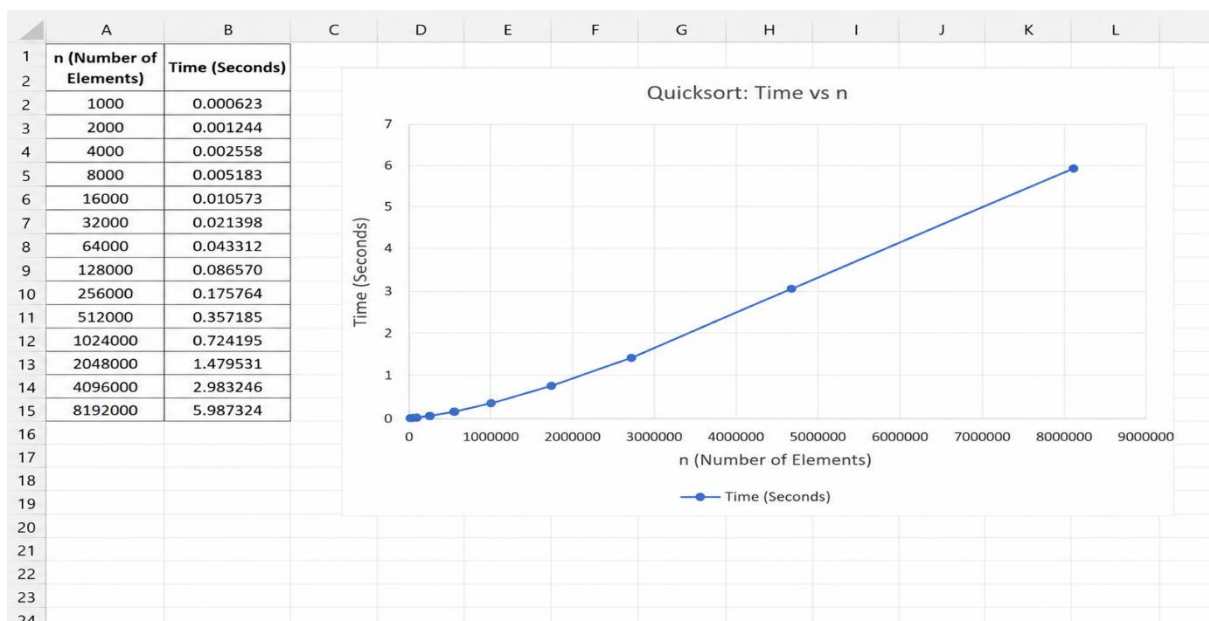
```

PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BPQ4CS088> cd "c:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BPQ4CS088\Lab-Program04-QuickSort\" ; if ($?) { gcc 01-QuickSort.c -o 01-QuickSort } ; if ($?) { .\01-QuickSort }
Enter the number of elements: 5
Enter the elements:
4 2 7 1 3
Sorted array:
1 2 3 4 7
PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BPQ4CS088\Lab-Program04-QuickSort>

```

## Time Complexity and Graph:

Time complexity:  $O(n \log n)$



## Lab Program 5: Heap Sort:

### Code:

```
#include <stdio.h>

void heapify(int a[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if(left < n && a[left] > a[largest])
        largest = left;

    if(right < n && a[right] > a[largest])
        largest = right;

    if(largest != i)
    {
        int temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;

        heapify(a, n, largest);
    }
}

void heapSort(int a[], int n)
{
    for(int i = n / 2 - 1; i >= 0; i--)
```



```

        heapify(a, n, i);

for(int i = n - 1; i > 0; i--)
{
    int temp = a[0];
    a[0] = a[i];
    a[i] = temp;

    heapify(a, i, 0);
}

int main()
{
    int n, a[100];

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    heapSort(a, n);

    printf("Sorted array:\n");
    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

```

```
printf("\n");
```

```
return 0;
```

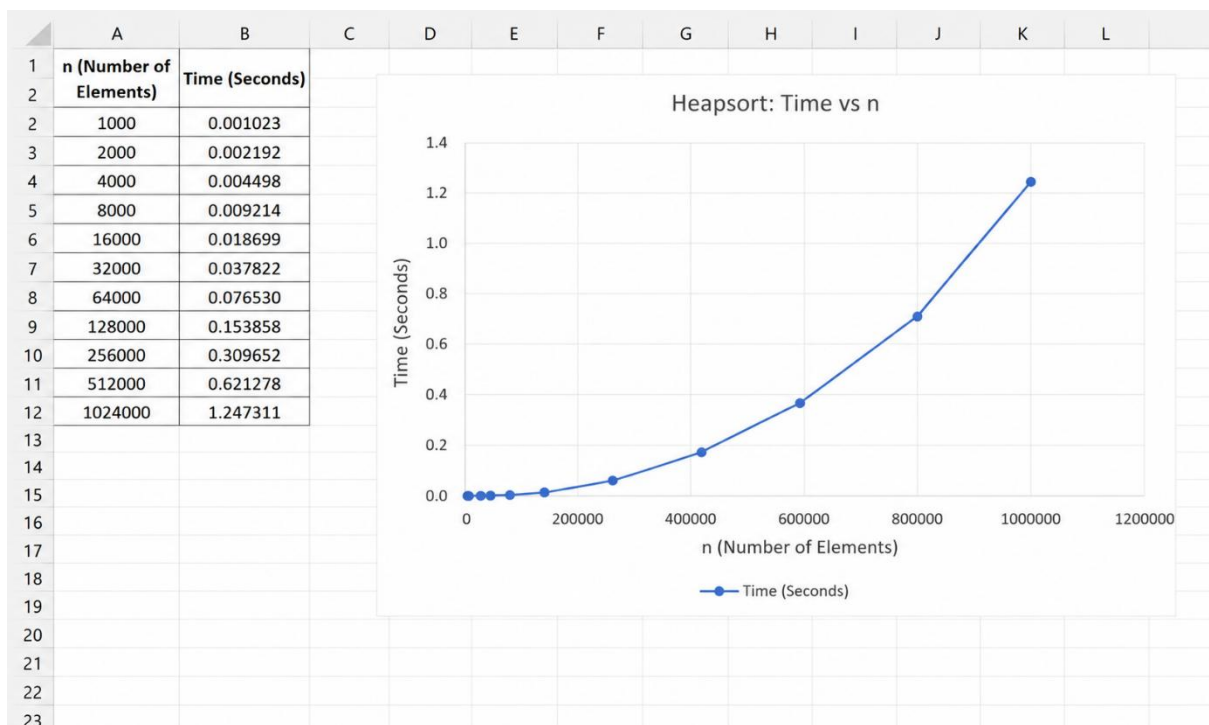
```
}
```

## Output:

```
PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BP24CS088> cd "c:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BP24CS088\ADA-Lab-1BP24CS088\Lab-Program10-Heap-Sort\" ; if ($?) { gcc 01-Heapsort.c -o 01-Heapsort } ; if ($?) { .\01-Heapsort }
Enter the number of elements: 6
Enter the elements:
45 12 78 34 23 90
Sorted arrays:
12 23 34 45 78 90
PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BP24CS088\ADA-Lab-1BP24CS088\Lab-Program10-Heap-Sort>
```

## Time Complexity and Graph:

Time complexity:-  $O(n \log n)$



### Lab Program 6:0/1 Knap-Sack:

#### Code:

```
#include <stdio.h>

int max(int a, int b)
{
    return (a > b) ? a : b;
}

int main()
{
    int n, W;

    printf("Enter the number of items: ");
    scanf("%d", &n);

    int profit[n], weight[n];

    printf("Enter profits:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &profit[i]);

    printf("Enter weights:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &weight[i]);

    printf("Enter knapsack capacity: ");
    scanf("%d", &W);
```

```

int dp[n + 1][W + 1];

for(int i = 0; i <= n; i++)
{
    for(int w = 0; w <= W; w++)
    {
        if(i == 0 || w == 0)
            dp[i][w] = 0;
        else if(weight[i - 1] <= w)
            dp[i][w] = max(profit[i - 1] + dp[i - 1][w - weight[i - 1]],
                           dp[i - 1][w]);
        else
            dp[i][w] = dp[i - 1][w];
    }
}

printf("Maximum profit = %d\n", dp[n][W]);

return 0;
}

```

### Output:

```

PS C:\Users\dd788\OneDrive\Documents\ADA-Lab-4thsem-1BP24CS088> cd "c:\Users\dd788\OneDrive\Documents\ADA-Lab-4thsem-1BP24CS088\Lab-Program06-0\knapsack\01-0\"; if ($?) { gcc 1knapsack.c -o 1knapsack }; if ($?) { .\1knapsack }
Enter the number of items: 5
Enter profits:
12
15
13
18
20
Enter weights:
12
45
20
10
5
Enter knapsack capacity: 5
Maximum profit = 20
PS C:\Users\dd788\OneDrive\Documents\ADA-Lab-4thsem-1BP24CS088\Lab-Program06-0\knapsack\01-0>

```

### Time Complexity and Graph:

Time Complexity:-  $O(nW)$

### Lab Program 7: Flyod's Algorithm:

#### Code:

```
#include <stdio.h>

#define INF 9999

int main()
{
    int n;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    int cost[n][n];

    printf("Enter the cost adjacency matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if(i != j && cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }

    for(int k = 0; k < n; k++)
    {
```

```

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            if(cost[i][k] + cost[k][j] < cost[i][j])
                cost[i][j] = cost[i][k] + cost[k][j];
        }
    }
}

printf("All-Pairs Shortest Path Matrix:\n");

for(int i = 0; i < n; i++)
{
    for(int j = 0; j < n; j++)
    {
        if(cost[i][j] == INF)
            printf("INF ");
        else
            printf("%d ", cost[i][j]);
    }
    printf("\n");
}

return 0;
}

```

## Output:

```
PROBLEMS OUTPUT TERMINAL PORTS SPELL CHECKER powershell - Lab-Program11-Floyd's Algorithm
PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BPQ4CS088> cd "c:\Users\dd708\OneDrive\Documents\ADA-Lab-1BPQ4CS088\Lab-Program11-Floyd's Algorithm"
>> gcc -std=c99 -Wall -Wextra -I.\01-Floyd's.c" -o floyd.exe
>> .\floyd.exe
Enter the number of vertices: 4
Enter the cost adjacency matrix:
0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0
All-Pairs Shortest Path Matrix:
0 5 8 9
999 0 3 4
999 999 0 1
999 999 999 0
PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BPQ4CS088\Lab-Program11-Floyd's Algorithm>
```

**Time complexity:-  $T(n)=O(n^3)$ .**

### Lab Program 8a:Prim's Algorithm:

#### Code:

```
#include <stdio.h>

#define INF 9999

int main()
{
    int n;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    int cost[n][n];

    printf("Enter the cost adjacency matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if(cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }

    int visited[n];

    for(int i = 0; i < n; i++)
```



```

        visited[i] = 0;

visited[0] = 1;

int edges = 0, minCost = 0;

printf("Edges in the Minimum Spanning Tree:\n");

while(edges < n - 1)
{
    int min = INF, a = -1, b = -1;

    for(int i = 0; i < n; i++)
    {
        if(visited[i])
        {
            for(int j = 0; j < n; j++)
            {
                if(!visited[j] && cost[i][j] < min)
                {
                    min = cost[i][j];
                    a = i;
                    b = j;
                }
            }
        }
    }

    printf("%d -> %d : %d\n", a, b, min);

```

```

        visited[b] = 1;

        minCost += min;

        edges++;

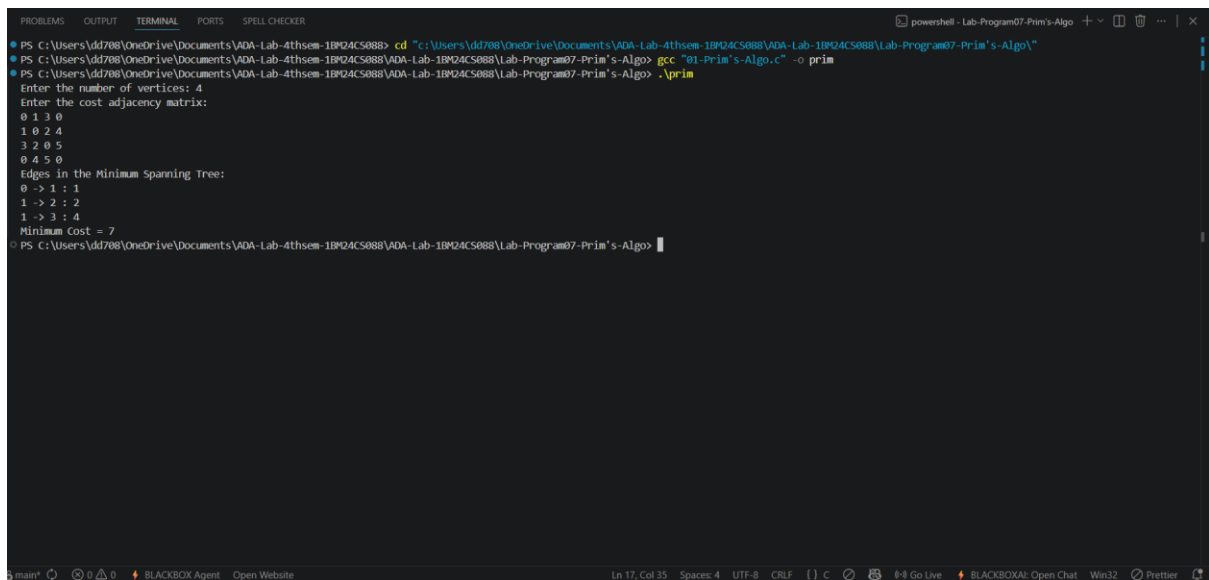
    }

    printf("Minimum Cost = %d\n", minCost);

    return 0;
}

```

### Output:



```

PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-18Q24CS088> cd "c:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-18Q24CS088\ADA-Lab-18Q24CS088\Lab-Program07-Prim's-Algo\"
PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-18Q24CS088\ADA-Lab-18Q24CS088\Lab-Program07-Prim's-Algo> gcc "01-Prim's-Algo.c" -o prim
PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-18Q24CS088\ADA-Lab-18Q24CS088\Lab-Program07-Prim's-Algo> .\prim
Enter the number of vertices: 4
Enter the cost adjacency matrix:
0 1 3 0
1 0 2 4
3 2 0 5
0 4 5 0
Edges in the Minimum Spanning Tree:
0 -> 1 : 1
1 -> 2 : 2
1 -> 3 : 4
Minimum Cost = 7
PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-18Q24CS088\ADA-Lab-18Q24CS088\Lab-Program07-Prim's-Algo>

```

**Time complexity:-  $T(n)=O(n^2)$**

### Lab Program 8b: Kruskal's Algorithm:

#### Code:

```
#include <stdio.h>

struct Edge
{
    int u, v, w;
};

int parent[100];

int find(int i)
{
    while(parent[i] != i)
        i = parent[i];
    return i;
}

void unionSet(int a, int b)
{
    parent[a] = b;
}

int main()
{
    int n;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);
```

```

int cost[n][n];

printf("Enter the cost adjacency matrix:\n");
for(int i = 0; i < n; i++)
{
    for(int j = 0; j < n; j++)
    {
        scanf("%d", &cost[i][j]);
    }
}

struct Edge edges[100];
int e = 0;

for(int i = 0; i < n; i++)
{
    for(int j = i + 1; j < n; j++)
    {
        if(cost[i][j] != 0)
        {
            edges[e].u = i;
            edges[e].v = j;
            edges[e].w = cost[i][j];
            e++;
        }
    }
}

```

```

for(int i = 0; i < e - 1; i++)
{
    for(int j = 0; j < e - i - 1; j++)
    {
        if(edges[j].w > edges[j + 1].w)
        {
            struct Edge temp = edges[j];
            edges[j] = edges[j + 1];
            edges[j + 1] = temp;
        }
    }
}

for(int i = 0; i < n; i++)
    parent[i] = i;

int minCost = 0, count = 0;

printf("Edges in the Minimum Spanning Tree:\n");

for(int i = 0; i < e && count < n - 1; i++)
{
    int a = find(edges[i].u);
    int b = find(edges[i].v);

    if(a != b)
    {
        printf("%d -> %d : %d\n", edges[i].u, edges[i].v, edges[i].w);
    }
}

```

```

        minCost += edges[i].w;

        unionSet(a, b);

        count++;
    }
}

printf("Minimum Cost = %d\n", minCost);

return 0;
}

```

### Output:

```

PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BP24CS088> cd ".\ADA-Lab-1BP24CS088\Lab-Program08-Kruskal's-Algo\"
PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BP24CS088\ADA-Lab-1BP24CS088\Lab-Program08-Kruskal's-Algo> gcc "01-Kruskal's-Algo.c" -o kruskal
>> .\kruskal
Enter the number of vertices: 4
Enter the cost adjacency matrix:
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0
Edges in the Minimum Spanning Tree:
2 -> 3 : 4
0 -> 3 : 5
0 -> 1 : 10
Minimum Cost = 19
PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BP24CS088\ADA-Lab-1BP24CS088\Lab-Program08-Kruskal's-Algo>

```

**Time Complexity:-  $T(E)=O(E^2)$**

### Lab Program 9: Fractional Knap-Sack Using Greedy Method:

#### Code:

```
#include <stdio.h>

struct Item
{
    int profit;
    int weight;
    float ratio;
};

int main()
{
    int n;
    float capacity, totalProfit = 0;

    printf("Enter the number of items: ");
    scanf("%d", &n);

    struct Item item[n];

    printf("Enter profit and weight of each item:\n");
    for(int i = 0; i < n; i++)
    {
        scanf("%d %d", &item[i].profit, &item[i].weight);
        item[i].ratio = (float)item[i].profit / item[i].weight;
    }

    printf("Enter the knapsack capacity: ");
```

```

scanf("%f", &capacity);

for(int i = 0; i < n - 1; i++)
{
    for(int j = 0; j < n - i - 1; j++)
    {
        if(item[j].ratio < item[j + 1].ratio)
        {
            struct Item temp = item[j];
            item[j] = item[j + 1];
            item[j + 1] = temp;
        }
    }
}

printf("Selected items:\n");

for(int i = 0; i < n; i++)
{
    if(capacity >= item[i].weight)
    {
        capacity -= item[i].weight;
        totalProfit += item[i].profit;
        printf("Item %d -> 100%% selected\n", i + 1);
    }
    else
    {
        float fraction = capacity / item[i].weight;
        totalProfit += item[i].profit * fraction;
    }
}

```



```

        printf("Item %d -> %.2f%% selected\n", i + 1, fraction * 100);

        break;

    }

}

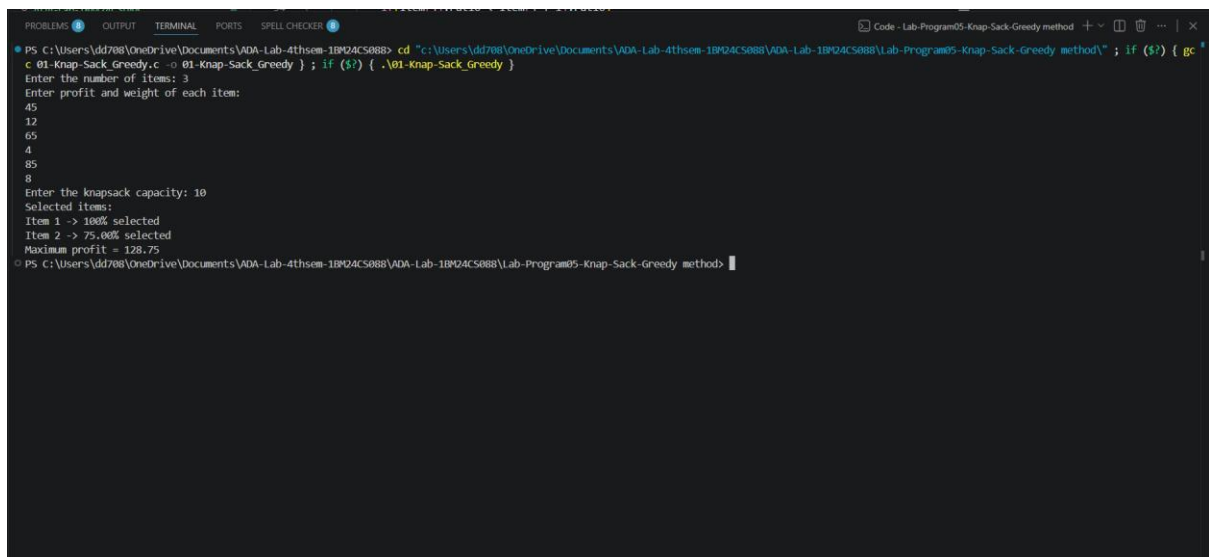
printf("Maximum profit = %.2f\n", totalProfit);

return 0;

}

```

### Output:



```

PS C:\Users\dd788\OneDrive\Documents\ADA-Lab-4thsem-18P04CS088> cd "c:\Users\dd788\OneDrive\Documents\ADA-Lab-4thsem-18P04CS088\ADA-Lab-18P04CS088\Lab-Program05-Knap-Sack-Greedy method\" ; if ($?) { gcc 01-Knap-Sack_Greedy.c -o 01-Knap-Sack_Greedy } ; if ($?) { .\01-Knap-Sack_Greedy }
Enter the number of items: 3
Enter profit and weight of each item:
45
12
65
4
85
8
Enter the knapsack capacity: 10
Selected items:
Item 1 -> 100% selected
Item 2 -> 75.00% selected
Maximum profit = 128.75
PS C:\Users\dd788\OneDrive\Documents\ADA-Lab-4thsem-18P04CS088\ADA-Lab-18P04CS088\Lab-Program05-Knap-Sack-Greedy method>

```

**Time complexity:**  $T(n)=O(n^2)$ .

## Lab Program 10:Dijkstra's Algorithm:

### Code:

```
#include <stdio.h>

#define INF 9999

int main()
{
    int n;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    int cost[n][n];

    printf("Enter the cost adjacency matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if(i != j && cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }

    int source;
```

```

printf("Enter the source vertex: ");
scanf("%d", &source);

int dist[n], visited[n];

for(int i = 0; i < n; i++)
{
    dist[i] = cost[source][i];
    visited[i] = 0;
}

dist[source] = 0;
visited[source] = 1;

for(int count = 1; count < n; count++)
{
    int min = INF, u = -1;

    for(int i = 0; i < n; i++)
    {
        if(!visited[i] && dist[i] < min)
        {
            min = dist[i];
            u = i;
        }
    }

    visited[u] = 1;

```

```

        for(int v = 0; v < n; v++)
        {
            if(!visited[v] && dist[u] + cost[u][v] < dist[v])
            {
                dist[v] = dist[u] + cost[u][v];
            }
        }
    }

    printf("Shortest distances from vertex %d:\n", source);

    for(int i = 0; i < n; i++)
    {
        printf("%d -> %d = %d\n", source, i, dist[i]);
    }

    return 0;
}

```

### Output:

```

PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-18Q4CS088> cd "C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-18Q4CS088\ADA-Lab-18Q4CS088\Lab-Program09-Dijkstra's-Algo"
>>
>> gcc -std=c99 -Wall -Wextra -c ".\01-Dijkstra's.c" -o dijkstra.exe
>>
>> .\dijkstra.exe
0 2 0 6
2 0 3 8
0 3 0 0
0 8 0 0
Enter the source vertex: 0
Shortest distances from vertex 0:
0 -> 0 = 0
0 -> 1 = 2
0 -> 2 = 5
0 -> 3 = 6
PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-18Q4CS088\ADA-Lab-18Q4CS088\Lab-Program09-Dijkstra's-Algo>

```

**Time complexity:-  $T(n)=O(n^2)$ .**

### Lab Program 11: N-Queens Backtrack:

#### Code:

```
#include <stdio.h>

#include <stdlib.h>

int x[20];

int place(int k, int i)
{
    for(int j = 1; j < k; j++)
    {
        if(x[j] == i || abs(x[j] - i) == abs(j - k))
            return 0;
    }
    return 1;
}

void nQueens(int k, int n)
{
    for(int i = 1; i <= n; i++)
    {
        if(place(k, i))
        {
            x[k] = i;

            if(k == n)
            {
                printf("\nSolution:\n");
            }
        }
    }
}
```

```

        for(int r = 1; r <= n; r++)
        {
            for(int c = 1; c <= n; c++)
            {
                if(x[r] == c)
                    printf("Q ");
                else
                    printf(". ");
            }
            printf("\n");
        }
    }
else
{
    nQueens(k + 1, n);
}
}
}
}
}

```

```

int main()
{
    int n;

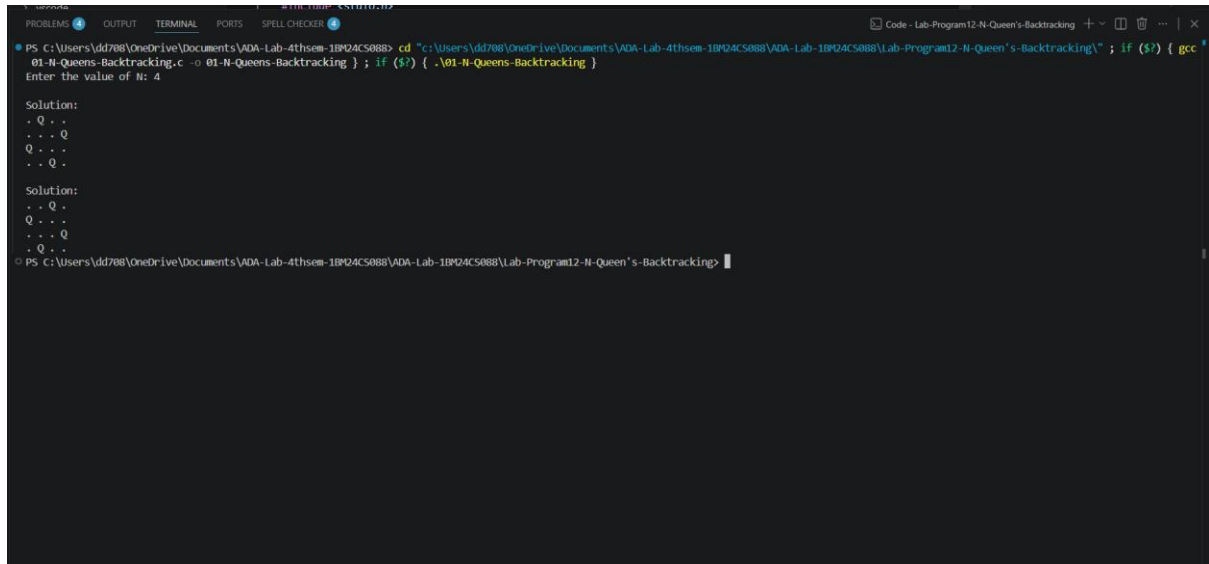
    printf("Enter the value of N: ");
    scanf("%d", &n);

    nQueens(1, n);
}

```

```
    return 0;
}
```

## Output:



```
PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BP24CS088> cd "c:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BP24CS088\ADA-Lab-1BP24CS088\Lab-Program12-Queen's-Backtracking\" ; if ($?) { gcc 01-N-Queens-Backtracking.c -o 01-N-Queens-Backtracking } ; if ($?) { .\01-N-Queens-Backtracking }
Enter the value of N: 4

Solution:
. Q . .
. . . Q
Q . . .
. . Q .

Solution:
. . Q .
Q . . .
. . . Q
. Q . .

PS C:\Users\dd708\OneDrive\Documents\ADA-Lab-4thsem-1BP24CS088\ADA-Lab-1BP24CS088\Lab-Program12-Queen's-Backtracking>
```

**Time complexity:-**  $T(N) = O(N!)$ .